

Flutter Floor Database Report

Application Report

Notes App — ORM Architecture with Floor

Flutter SDK ^3.11.5 • floor ^1.5.0 • sqflite ^2.4.2

Package	room_database
Database File	room_database.db
Report Date	Jun 1, 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary	4
2. Project Overview	5
2.1 Application Purpose.....	5
2.2 Learning Objectives Demonstrated	5
2.3 Project Structure.....	5
3. Database Architecture.....	7
3.1 The Floor ORM Pattern	7
3.2 Database Class	7
3.3 Schema Design	7
4. Entity Classes	9
4.1 NotebookEntity	9
4.2 NoteEntity.....	9
5. Data Access Objects (DAOs)	11
5.1 NoteDao	11
5.2 NotebookDao	11
6. Repository Layer	13
6.1 Architecture Design	13
6.2 FloorNotesRepository	13
7. UI Layer	15
7.1 App Entry Point.....	15
7.2 Reactive List with StreamBuilder	15
7.3 Note Editor	15
7.4 UI Features.....	15
8. Code Generation with build_runner.....	17
8.1 How It Works	17
8.2 Running the Generator	17
8.3 Generated Code Highlights.....	17
9. CRUD Operations	18
9.1 Create	18
9.2 Read.....	18
9.3 Update.....	18
9.4 Delete.....	18
10. Floor vs sqflite Comparison.....	19
10.1 When to Use Floor.....	19

10.2 When to Use Raw sqflite	19
10.3 Performance Considerations	20
11. Setup Guide (README).....	21
11.1 Prerequisites.....	21
11.2 Installation	21
11.3 Running the App.....	21
11.4 Regenerating After Schema Changes	21
12. Key Takeaways.....	22
12.1 ORM Benefits Observed in This App	22
12.2 ORM Trade-offs Observed.....	22
12.3 Architectural Patterns Worth Noting.....	22
13. Conclusion	22

1. Executive Summary

This report documents the Flutter Notes application built using the Floor ORM library as the local database layer. The app demonstrates a complete implementation of Floor as a type-safe, annotation-driven alternative to raw sqlite, with a clean architecture that separates data concerns from UI logic.

The application implements a Notes management system with notebook-level organisation, full CRUD operations, and reactive UI updates via Dart Streams. The codebase serves as a practical reference for developers choosing between Floor and raw sqlite for Flutter persistence.

App Name	Floor Notes (room_database package)
Flutter SDK	^3.11.5 (Dart SDK ^3.11.5)
Floor Version	^1.5.0
sqlite Version	^2.4.2
Database File	room_database.db (persistent on device)
Architecture	Repository pattern with Floor ORM
UI Framework	Material 3 / Flutter Widgets
Code Generation	build_runner ^2.4.13 + floor_generator ^1.5.0

2. Project Overview

2.1 Application Purpose

The app is a Notes application that allows users to create, read, update, and delete personal notes. Notes are organised within named notebooks, and the default notebook ("Daily Notes") is automatically seeded on first launch. The interface uses Material 3 design with a teal-green colour scheme.

2.2 Learning Objectives Demonstrated

- Understanding Floor as an ORM (Object-Relational Mapping) pattern for Flutter
- Setting up Floor with the `@Database`, `@Entity`, and `@dao` annotations
- Creating type-safe DAOs (Data Access Objects) with `@Query`, `@Insert`, `@Update`, and `@Delete`
- Using `build_runner` to generate Floor implementation code
- Implementing reactive data queries using Dart Streams
- Managing foreign key relationships between entities
- Comparing ORM abstractions with raw `sqlite` SQL

2.3 Project Structure

File	Purpose
<code>lib/main.dart</code>	App entry point; initialises Floor database and seeds default notebook
<code>lib/app.dart</code>	Root widget, home page, note list, note editor bottom sheet
<code>lib/data/app_database.dart</code>	Floor <code>@Database</code> class declaring entities and DAO accessors
<code>lib/data/app_database.g.dart</code>	Auto-generated Floor implementation (never edit manually)
<code>lib/data/note_entity.dart</code>	<code>NoteEntity</code> <code>@Entity</code> with foreign key to <code>NotebookEntity</code>
<code>lib/data/notebook_entity.dart</code>	<code>NotebookEntity</code> <code>@Entity</code> — parent table for notes
<code>lib/data/note_dao.dart</code>	<code>NoteDao</code> <code>@dao</code> — CRUD + Stream queries for notes
<code>lib/data/notebook_dao.dart</code>	<code>NotebookDao</code> <code>@dao</code> — insert and lookup for notebooks
<code>lib/data/notes_repository.dart</code>	Abstract <code>NotesRepository</code> interface
<code>lib/data/floor_notes_repository.dart</code>	Concrete Floor-backed implementation of <code>NotesRepository</code>
<code>lib/data/in_memory_notes_repository.dart</code>	In-memory implementation for testing/previews

3. Database Architecture

3.1 The Floor ORM Pattern

Floor is Flutter's equivalent to Android's Room database library. It provides an abstraction layer over sqflite using Dart annotations and code generation to produce type-safe database access code. The developer defines the schema declaratively using annotations, and `build_runner` generates the boilerplate at compile time.

The three core components of any Floor app are:

- Entities — Plain Dart classes annotated with `@Entity` that map to database tables
- DAOs (Data Access Objects) — Abstract classes annotated with `@dao` containing query method signatures
- Database — An abstract class annotated with `@Database` that ties entities and DAOs together

3.2 Database Class

The `AppDatabase` class is the entry point for all database access:

```
@Database(version: 1, entities: [NotebookEntity, NoteEntity])
abstract class AppDatabase extends FloorDatabase {
  NotebookDao get notebookDao;
  NoteDao get noteDao;
}
```

The `@Database` annotation registers both entities and sets the schema version. Floor generates the concrete `_AppDatabase` class and the `$FloorAppDatabase` builder that is used in `main.dart` to construct the database:

```
final database = await $FloorAppDatabase
  .databaseBuilder('room_database.db')
  .build();
```

3.3 Schema Design

The database contains two tables with a parent-child relationship:

Column	Type	Constraint	Notes
— notebooks table —			
id	INTEGER	PRIMARY KEY	Manual, not auto-generated
name	TEXT	NOT NULL	Notebook display name

Column	Type	Constraint	Notes
<code>created_at</code>	INTEGER	NOT NULL	Unix millis timestamp
— notes table —			
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-generated
<code>notebook_id</code>	INTEGER	FOREIGN KEY (notebooks.id)	ON DELETE CASCADE
<code>title</code>	TEXT	NOT NULL	Note heading
<code>content</code>	TEXT	NOT NULL	Note body
<code>created_at</code>	INTEGER	NOT NULL	Unix millis (immutable after creation)
<code>updated_at</code>	INTEGER	NOT NULL	Updated on every save

4. Entity Classes

4.1 NotebookEntity

The NotebookEntity class maps to the notebooks table and acts as the parent in the foreign key relationship with notes:

```
@Entity(tableName: 'notebooks')
class NotebookEntity {
  @PrimaryKey(autoGenerate: false)
  final int id;

  final String name;

  @ColumnInfo(name: 'created_at')
  final int createdAtMillis;
}
```

Notable design decisions:

- `autoGenerate: false` — the ID is manually assigned (value 1 for the seeded notebook)
- `@ColumnInfo(name: 'created_at')` — remaps the Dart camelCase field to a snake_case column

4.2 NoteEntity

NoteEntity is more complex, using a foreign key with cascade delete and an index on notebook_id for query performance:

```
@Entity(
  tableName: 'notes',
  foreignKeys: [
    ForeignKey(
      childColumns: ['notebook_id'],
      parentColumns: ['id'],
      entity: NotebookEntity,
      onDelete: ForeignKeyAction.cascade,
    ),
  ],
  indices: [Index(value: ['notebook_id'])],
)
```

Key fields:

- `id` — int? (nullable) with `autoGenerate: true`; Floor treats null as insert-new
- `notebookId` — mapped to 'notebook_id' column via `@ColumnInfo`
- `createdAtMillis` / `updatedAtMillis` — stored as INTEGER (epoch milliseconds)

The entity also provides a `copyWith()` helper for immutable updates — a pattern that pairs well with Flutter's reactive state management:

```
final updated = note.copyWith(title: 'New Title',  
    updatedAtMillis: DateTime.now().millisecondsSinceEpoch);
```

5. Data Access Objects (DAOs)

5.1 NoteDao

The NoteDao interface defines all data operations for notes. Floor generates the concrete implementation at build time:

```
@dao
abstract class NoteDao {
    @Query('SELECT * FROM notes WHERE notebook_id = :notebookId
          ORDER BY updated_at DESC, id DESC')
    Stream<List<NoteEntity>> watchNotesForNotebook(int notebookId);

    @Query('SELECT * FROM notes WHERE id = :id LIMIT 1')
    Future<NoteEntity?> findNoteById(int id);

    @Insert(onConflict: OnConflictStrategy.replace)
    Future<int> insertNote(NoteEntity note);

    @Update()
    Future<int> updateNote(NoteEntity note);

    @Query('DELETE FROM notes WHERE id = :id')
    Future<void> deleteNoteById(int id);
}
```

Key observations about NoteDao:

- watchNotesForNotebook returns a Stream — Floor automatically emits new values whenever the notes table changes, enabling reactive UI without manual polling
- insertNote uses OnConflictStrategy.replace — equivalent to SQL INSERT OR REPLACE
- updateNote uses the @Update annotation which generates an UPDATE WHERE id = ? statement automatically
- deleteNoteById uses @Query rather than @Delete, giving precise control over the WHERE clause

5.2 NotebookDao

The NotebookDao is simpler, providing just lookup and insert:

```
@dao
abstract class NotebookDao {
    @Query('SELECT * FROM notebooks WHERE id = :id LIMIT 1')
    Future<NotebookEntity?> findNotebookById(int id);
}
```

```
@Insert(onConflict: OnConflictStrategy.replace)
Future<void> insertNotebook(NotebookEntity notebook);
}
```

This minimal DAO is used only for seeding the default notebook on first launch and for verifying it already exists.

6. Repository Layer

6.1 Architecture Design

The app uses the Repository pattern to decouple the UI from the database implementation. The abstract `NotesRepository` interface defines the contract:

```
abstract class NotesRepository {
  Stream<List<NoteEntity>> watchNotes();
  Future<NoteEntity?> findNoteById(int id);
  Future<void> saveNote({NoteEntity? existing,
    required String title, required String content});
  Future<void> deleteNote(NoteEntity note);
  Future<void> seed();
}
```

Two implementations exist:

- `FloorNotesRepository` — the production Floor-backed implementation
- `InMemoryNotesRepository` — a lightweight in-memory implementation used for testing and preview

6.2 FloorNotesRepository

The concrete implementation wraps the `AppDatabase` and applies business logic such as:

- Defaulting all notes to a fixed notebook ID (1) — the hardcoded 'Daily Notes' notebook
- Setting timestamps automatically in `saveNote()` rather than requiring the caller to provide them
- Routing to `insertNote` or `updateNote` based on whether `existing` is null
- Null-checking the note ID before calling `deleteNoteById`

The `saveNote()` method demonstrates an important Floor pattern — using the entity's nullable id field to distinguish inserts from updates:

```
final note = NoteEntity(
  id: existing?.id, // null = new insert; non-null = update
  notebookId: _defaultNotebookId,
  title: title.trim(),
  ...
);
if (existing == null) {
  await _database.noteDao.insertNote(note);
} else {
  await _database.noteDao.updateNote(note);
}
```


7. UI Layer

7.1 App Entry Point

main.dart is responsible for initialising the database before the widget tree is built. The async main() function ensures the database and default notebook are ready before runApp() is called:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final database = await $FloorAppDatabase
    .databaseBuilder('room_database.db').build();
  final repository = FloorNotesRepository(database);
  await repository.seed();
  runApp(MyApp(repository: repository));
}
```

7.2 Reactive List with StreamBuilder

The notes list is powered by a StreamBuilder that subscribes to watchNotes(). Because Floor's Stream emits a new value on any table change, the list refreshes automatically after insert, update, or delete — no setState() or explicit refresh needed:

```
StreamBuilder<List<NoteEntity>>(<
  stream: widget.repository.watchNotes(),
  builder: (context, snapshot) {
    final notes = snapshot.data ?? const <NoteEntity>[];
    // build list from notes...
  },
)
```

7.3 Note Editor

The note editor is presented as a modal bottom sheet with two TextFields (title and content). On save, it calls repository.saveNote() and pops with true to indicate success. The home page listens for this return value to show a SnackBar confirmation.

7.4 UI Features

- Material 3 theming with teal seed colour
- Gradient background (light blue-green) for visual depth
- Collapsible SliverAppBar showing 'Floor Notes' title
- Header statistics card showing total note count
- Empty state widget when no notes exist
- Note cards with title, content preview, timestamps, and edit/delete actions
- SnackBar confirmations for save, update, and delete operations

8. Code Generation with build_runner

8.1 How It Works

Floor relies on Dart's source_gen / build_runner toolchain to generate the concrete database implementation. The generated file (app_database.g.dart) contains:

- The \$FloorAppDatabase class with static builder methods
- The _\$AppDatabaseBuilder class implementing the builder pattern
- The _\$AppDatabase class extending AppDatabase with actual SQL execution
- Concrete DAO implementations for NoteDao and NotebookDao

8.2 Running the Generator

The generator is run once after any changes to entities or DAOs:

```
flutter pub run build_runner build
```

For continuous generation during development:

```
flutter pub run build_runner watch
```

8.3 Generated Code Highlights

The generated app_database.g.dart creates:

- A CREATE TABLE SQL statement derived directly from each @Entity class
- A query method for each @Query-annotated DAO method, with named parameters bound from method arguments
- A change listener using Dart's StreamController.broadcast() to power the Stream-returning DAO methods
- Automatic JSON-like mapping from sqlite Map<String, Object?> rows to entity constructors

9. CRUD Operations

9.1 Create

A new note is created by calling `saveNote()` with no existing parameter. The repository constructs a `NoteEntity` with `id`: null (which Floor treats as insert-new) and calls `insertNote()`:

- Floor generates: `INSERT OR REPLACE INTO notes (notebook_id, title, ...) VALUES (?, ?, ...)`
- Return value is the new row ID as int
- The Stream from `watchNotesForNotebook` automatically emits the updated list

9.2 Read

Two read patterns are used:

- `watchNotesForNotebook(int notebookId)` — returns `Stream<List<NoteEntity>>`, used by the home screen for reactive updates
- `findNoteById(int id)` — returns `Future<NoteEntity?>`, used when a single note needs to be loaded

9.3 Update

Updating calls `saveNote()` with the existing `NoteEntity`. The repository preserves the original `createdAtMillis` and updates `updatedAtMillis` to now:

- Floor generates: `UPDATE notes SET notebook_id=?, title=?, ... WHERE id=?`
- Return value is the number of rows affected (1 on success)

9.4 Delete

Deletion calls `deleteNoteById(int id)` which uses a `@Query` annotation:

- Floor generates: `DELETE FROM notes WHERE id = ?`
- The cascade foreign key means deleting a notebook would also delete all its notes automatically

10. Floor vs sqflite Comparison

A primary learning objective was to compare Floor (ORM) with raw sqflite. The table below summarises the key differences:

Aspect	Floor (ORM)	Raw sqflite
Setup complexity	Higher — requires annotations, code generation, build_runner	Lower — just add sqflite and write SQL
Type safety	Full compile-time type safety via generated code	Manual mapping; runtime errors possible
Query style	Annotated methods with @Query; SQL strings validated at build time	Raw SQL strings passed to database.query()
Boilerplate	Less — DAO interfaces auto-generate implementations	More — manual toMap()/fromMap() required
Reactive queries	Built-in Stream<List<T>> support via change listeners	Manual polling or stream wrappers needed
Migrations	Migration objects with version tracking in @Database	Manual onCreate/onUpgrade callbacks
Foreign keys	Declarative via ForeignKey annotation on entity	Manual SQL with PRAGMA foreign_keys=ON
Performance	Equivalent — Floor is a thin wrapper over sqflite	Slightly faster for very simple cases
Learning curve	Steeper; requires understanding ORM patterns and build_runner	Gentler; requires raw SQL knowledge
Best for	Complex apps, team projects, type-safety focus	Simple apps, prototyping, full SQL control

10.1 When to Use Floor

- Complex data models with multiple related entities
- Team projects requiring type safety and self-documenting code
- Apps needing reactive queries (Stream-based UI)
- Projects that need to evolve the schema over time with tracked migrations
- Developers who prefer convention-over-configuration

10.2 When to Use Raw sqflite

- Simple single-table apps or prototypes
- When build_runner adds too much CI complexity
- Scenarios requiring highly optimised or non-standard SQL
- Apps where full control of the SQL is a requirement
- Developers already comfortable with raw SQL who find ORMs opaque

10.3 Performance Considerations

Floor is a thin wrapper over sqflite and introduces negligible overhead for typical CRUD operations. Both approaches ultimately execute the same SQLite queries under the hood. The main overhead with Floor is the initial code generation step (build time only) and the change-listener infrastructure for Stream support. At runtime, query performance is essentially identical.

11. Setup Guide (README)

11.1 Prerequisites

- Flutter SDK $\geq$ 3.11.5 and Dart $\geq$ 3.11.5
- An IDE with Dart support (Android Studio, VS Code)

11.2 Installation

1. Add dependencies to pubspec.yaml:

```
dependencies:  
  floor: ^1.5.0  
  sqflite: ^2.4.2  
  
dev_dependencies:  
  build_runner: ^2.4.13  
  floor_generator: ^1.5.0
```

2. Fetch packages:

```
flutter pub get
```

3. Create entity classes with @Entity annotations
4. Create DAO abstract classes with @dao annotation
5. Create the @Database abstract class
6. Add 'part app_database.g.dart' to the database file
7. Run code generation:

```
flutter pub run build_runner build --delete-conflicting-outputs
```

11.3 Running the App

```
flutter run
```

11.4 Regenerating After Schema Changes

Any time you modify an entity, DAO, or database class, regenerate the code:

```
flutter pub run build_runner build --delete-conflicting-outputs
```

12. Key Takeaways

12.1 ORM Benefits Observed in This App

- Zero-boilerplate entity mapping — `NoteEntity` fields map directly to columns without manual `Map<String, Object?>` wiring
- Type-safe queries — passing the wrong type to a `@Query` method is caught at compile time
- Reactive by default — the Stream-based DAO method eliminated the need for any manual refresh logic in the UI
- Relationship integrity — the `ForeignKey` annotation with cascade delete handled referential integrity declaratively

12.2 ORM Trade-offs Observed

- Build-time dependency — changes to any annotated file require running `build_runner` before hot-reload takes effect
- Generated code opacity — debugging requires understanding the generated `_$.` classes
- Annotation learning curve — `@Entity`, `@ColumnInfo`, `ForeignKey`, `@dao`, `@Query`, `@Insert`, `@Update` are all new concepts

12.3 Architectural Patterns Worth Noting

- Repository abstraction — the `NotesRepository` interface let the UI be tested with `InMemoryNotesRepository` without a real database
- Nullable id convention — using `int?` for the primary key elegantly distinguishes insert from update at the model level
- Seed on startup — idempotent `seed()` method safely handles both first-run and subsequent launches
- Epoch millis for timestamps — storing timestamps as `INTEGER` milliseconds avoids SQLite date parsing complexity

13. Conclusion

The Floor Notes app successfully demonstrates all key learning objectives for Flutter's Floor ORM library. The implementation shows a clean, production-style architecture where entities, DAOs, the repository layer, and the UI each have well-defined responsibilities.

Floor is the right choice for this style of app: a multi-entity schema with relationships, reactive queries, and a codebase intended to grow over time. The investment in annotations and code generation pays dividends in type safety, reduced boilerplate, and built-in reactivity — all of which would require significant manual work with raw `sqlite`.

For developers coming from an Android background, Floor's API will be immediately familiar as a faithful Dart port of Room. For developers new to ORM patterns, this Notes app provides a minimal but complete reference implementation covering the full lifecycle: setup, entity definition, DAO design, code generation, and reactive UI integration.

Generated from source code analysis of room_database_app • May 29, 2026